

INFORME TÉCNICO

IMPLEMENTACIÓN DE LA BASE DE DATOS

“GradosTitulos” EN SQL SERVER

Documentación del procedimiento de creación, ejecución e implementación del script SQL

Estudiante: Chancasanampa Garcia Jeremy

Curso: Base de datos 2

Docente: _____

Fecha: _____

1. Introducción

El presente informe técnico documenta el procedimiento utilizado para implementar la base de datos “GradosTitulos” en Microsoft SQL Server. Se describe el proceso desde la preparación de la base de datos y la ejecución del script mediante SQL Server Management Studio (SSMS), hasta la creación de esquemas, tablas, relaciones, restricciones, datos iniciales, índices, vistas, funciones y procedimientos almacenados.

Además, se incorpora el código SQL utilizado para realizar la implementación, de manera que el procedimiento pueda ser reproducido y comprobado en un entorno SQL Server.

2. Objetivos

2.1 Objetivo general

Documentar técnicamente la implementación de la base de datos GradosTitulos en SQL Server, incluyendo el código utilizado para su creación y la verificación de los objetos generados.

2.2 Objetivos específicos

- Preparar la base de datos para ejecutar correctamente el script.
- Crear los esquemas y objetos que conforman la solución.
- Registrar los datos iniciales definidos en el script.
- Implementar índices, vistas, funciones y procedimientos almacenados.
- Verificar mediante consultas que los objetos hayan sido creados correctamente.

3. Herramientas utilizadas

- Microsoft SQL Server.
- SQL Server Management Studio (SSMS).
- Lenguaje Transact-SQL (T-SQL).
- Archivo de script SQL proporcionado para la base de datos GradosTitulos.

4. Procedimiento para insertar/implementar la base de datos en SQL Server

La implementación se realiza ejecutando el script en SQL Server Management Studio. El archivo proporcionado comienza indicando `USE GradosTitulos;`, por lo que la base de datos debe existir antes de ejecutar el resto del script. Para evitar el error “Database 'GradosTitulos' does not exist”, se recomienda crearla previamente si todavía no existe.

4.1 Paso 1: Crear la base de datos

Código utilizado para preparar la base de datos antes de ejecutar el script:

```
IF DB_ID('GradosTitulos') IS NULL
BEGIN
    CREATE DATABASE GradosTitulos;
END;
GO

USE GradosTitulos;
GO
```

Este bloque comprueba si la base de datos existe. Si no existe, la crea y posteriormente establece GradosTitulos como la base de datos activa mediante USE.

4.2 Paso 2: Abrir SQL Server Management Studio

- Abrir SQL Server Management Studio.
- Conectarse a la instancia de SQL Server.
- Seleccionar New Query / Nueva consulta.
- Pegar el script SQL proporcionado.
- Ejecutar primero el bloque de creación de la base de datos mostrado en el apartado anterior.
- Luego ejecutar el resto del script por bloques o en el orden indicado por el archivo.

4.3 Paso 3: Seleccionar la base de datos

El script original inicia con la instrucción:

```
USE GradosTitulos;
```

Esta instrucción indica que las operaciones posteriores deben realizarse dentro de la base de datos GradosTitulos. En el archivo fuente, esta instrucción aparece al inicio del script.

4.4 Paso 4: Crear los esquemas

El script crea cinco esquemas para organizar los objetos de la base de datos:

- Catalogo
- Academico
- Tramite
- Evaluacion
- Documento

Código del bloque de creación de esquemas:

```
CREATE SCHEMA Catalogo;
GO

CREATE SCHEMA Academico;
GO

CREATE SCHEMA Tramite;
GO

CREATE SCHEMA Evaluacion;
GO

CREATE SCHEMA Documento;
GO
```

4.5 Paso 5: Crear las tablas

Después de preparar los esquemas, se ejecutan las instrucciones CREATE TABLE del script. El archivo contiene 25 tablas distribuidas entre los cinco esquemas. Las tablas incorporan claves primarias, claves foráneas, restricciones CHECK, restricciones UNIQUE y valores DEFAULT, según corresponda.

Ejemplo del código utilizado para crear una tabla:

```
CREATE TABLE Catalogo.TipoTramite
(
    IdTipoTramite INT IDENTITY(1,1) NOT NULL,
    Nombre VARCHAR(100) NOT NULL,
    CONSTRAINT PK_TipoTramite PRIMARY KEY (IdTipoTramite),
    CONSTRAINT UQ_TipoTramite_Nombre UNIQUE (Nombre)
);
GO
```

El código anterior es un ejemplo representativo del tipo de definición utilizado en el script: se declara la estructura de la tabla y se establecen sus restricciones.

4.6 Paso 6: Insertar los datos iniciales

El script también contiene instrucciones INSERT para cargar información inicial en las tablas de catálogo y académicas. Entre ellas se encuentran TipoTramite, EstadoTramite y Facultad.

```
INSERT INTO Catalogo.TipoTramite (Nombre)
VALUES
('Grado de Bachiller'),
('Título Profesional'),
('Segunda Especialidad'),
('Duplicado');

INSERT INTO Catalogo.EstadoTramite (Nombre)
VALUES
('Registrado'),
('En revisión'),
('Observado'),
('Aprobado'),
('Rechazado'),
('En proceso'),
('Finalizado'),
('Anulado');
```

4.7 Paso 7: Crear índices

Posteriormente se ejecutan las instrucciones CREATE INDEX definidas en el script. Los índices se aplican, entre otros objetos, sobre Persona, Docente, ProgramaEstudios, Matricula, Tramite, TrabajoInvestigacion, EvaluacionTrabajo, Sustentacion y Resolucion.

```
-- Ejemplo de creación de índice
CREATE INDEX IX_Persona_Documento
ON Academico.Persona(Documento);
GO
```

Los nombres y columnas exactas de los índices deben ejecutarse tal como aparecen en el script original.

4.8 Paso 8: Crear vistas

El script crea las siguientes tres vistas:

- Academico.V_CondicionEgresado
- Tramite.V_EstadoTrabajo
- Evaluacion.V_NotaFinalSustentacion

```
CREATE VIEW Academico.V_CondicionEgresado
AS
```

```
-- consulta definida en el script original
SELECT ...
GO
```

La vista permite consultar información preparada a partir de una o más tablas sin repetir la consulta completa cada vez. En la implementación real debe ejecutarse la definición completa incluida en el archivo fuente.

4.9 Paso 9: Crear funciones

El script crea dos funciones:

- Academico.F_CicloMinimoPlan
- Evaluacion.F_SimilitudAceptable

Ejemplo de la estructura de ejecución de una función:

```
CREATE FUNCTION Academico.F_CicloMinimoPlan
(
    -- parámetros definidos en el script
)
RETURNS INT
AS
BEGIN
    -- lógica definida en el script
    RETURN ...;
END;
GO
```

4.10 Paso 10: Crear procedimientos almacenados

El script crea dos procedimientos almacenados:

- Tramite.SP_RegistrarTramite
- Evaluacion.SP_RegistrarSustentacion

```
CREATE PROCEDURE Tramite.SP_RegistrarTramite
    -- parámetros definidos en el script
AS
BEGIN
    -- lógica definida en el script
END;
GO
```

Los procedimientos almacenados encapsulan operaciones de negocio y permiten ejecutarlas mediante una llamada controlada desde SQL Server.

5. Código de ejecución en SQL Server

Para documentar de forma clara cómo se insertó la base de datos en SQL Server, el procedimiento general de ejecución puede representarse con el siguiente bloque:

```
-- 1. Crear la base de datos si no existe
IF DB_ID('GradosTitulos') IS NULL
BEGIN
    CREATE DATABASE GradosTitulos;
END;
GO
```

```
-- 2. Seleccionar la base de datos
USE GradosTitulos;
GO

-- 3. Ejecutar el script de implementación
-- Crear esquemas
-- Crear tablas
-- Insertar datos
-- Crear índices
-- Crear vistas
-- Crear funciones
-- Crear procedimientos almacenados
-- Ejecutar consultas de verificación
GO
```

En la práctica, los bloques siguientes se ejecutan desde una ventana de consulta de SSMS respetando el orden de dependencias del archivo original.

6. Verificación de la implementación

Al finalizar la ejecución, el script incluye consultas para revisar los objetos creados. El archivo también informa como resultado esperado: 25 tablas, 3 vistas, 2 funciones, 2 procedimientos almacenados y 5 esquemas.

```
-- Verificar tablas
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE'
ORDER BY TABLE_SCHEMA, TABLE_NAME;
GO

-- Verificar vistas
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.VIEWS
ORDER BY TABLE_SCHEMA, TABLE_NAME;
GO

-- Verificar procedimientos almacenados
SELECT SCHEMA_NAME(schema_id) AS Esquema,
       name AS Procedimiento
FROM sys.procedures
ORDER BY Esquema, Procedimiento;
GO

-- Verificar funciones
SELECT SCHEMA_NAME(schema_id) AS Esquema,
       name AS Funcion
FROM sys.objects
WHERE type IN ('FN','IF','TF')
ORDER BY Esquema, Funcion;
GO
```

7. Resultado esperado

- Base de datos: GradosTitulos.

- Esquemas: 5.
- Tablas: 25.
- Vistas: 3.
- Funciones: 2.
- Procedimientos almacenados: 2.

Estos valores corresponden al resumen final indicado por el script proporcionado. La comprobación debe realizarse en SQL Server después de ejecutar el archivo.

8. Evidencias que se deben colocar en el informe

- Captura de SQL Server Management Studio mostrando la conexión a la instancia.
- Captura de la ventana de consulta con el código de creación de GradosTitulos.
- Captura del Explorador de objetos mostrando la base de datos GradosTitulos.
- Captura de los esquemas Catalogo, Academico, Tramite, Evaluacion y Documento.
- Captura de las tablas creadas.
- Captura de las vistas, funciones y procedimientos almacenados.
- Captura de la ejecución correcta del script y del mensaje de finalización.
- Captura de las consultas de verificación.

9. Conclusiones

La base de datos GradosTitulos fue estructurada mediante un script T-SQL que organiza sus objetos en cinco esquemas y define las tablas, restricciones, datos iniciales, índices, vistas, funciones y procedimientos almacenados necesarios para su funcionamiento. La documentación del código permite reproducir el proceso de implementación en SQL Server y facilita la comprobación de los objetos creados.

10. Recomendaciones

- Crear la base de datos antes de ejecutar USE GradosTitulos si aún no existe.
- Ejecutar el script respetando el orden de creación de objetos y sus dependencias.
- Revisar la ventana Messages de SSMS para detectar errores de ejecución.
- Realizar las consultas de verificación al finalizar.
- Guardar una copia del script original antes de realizar modificaciones.

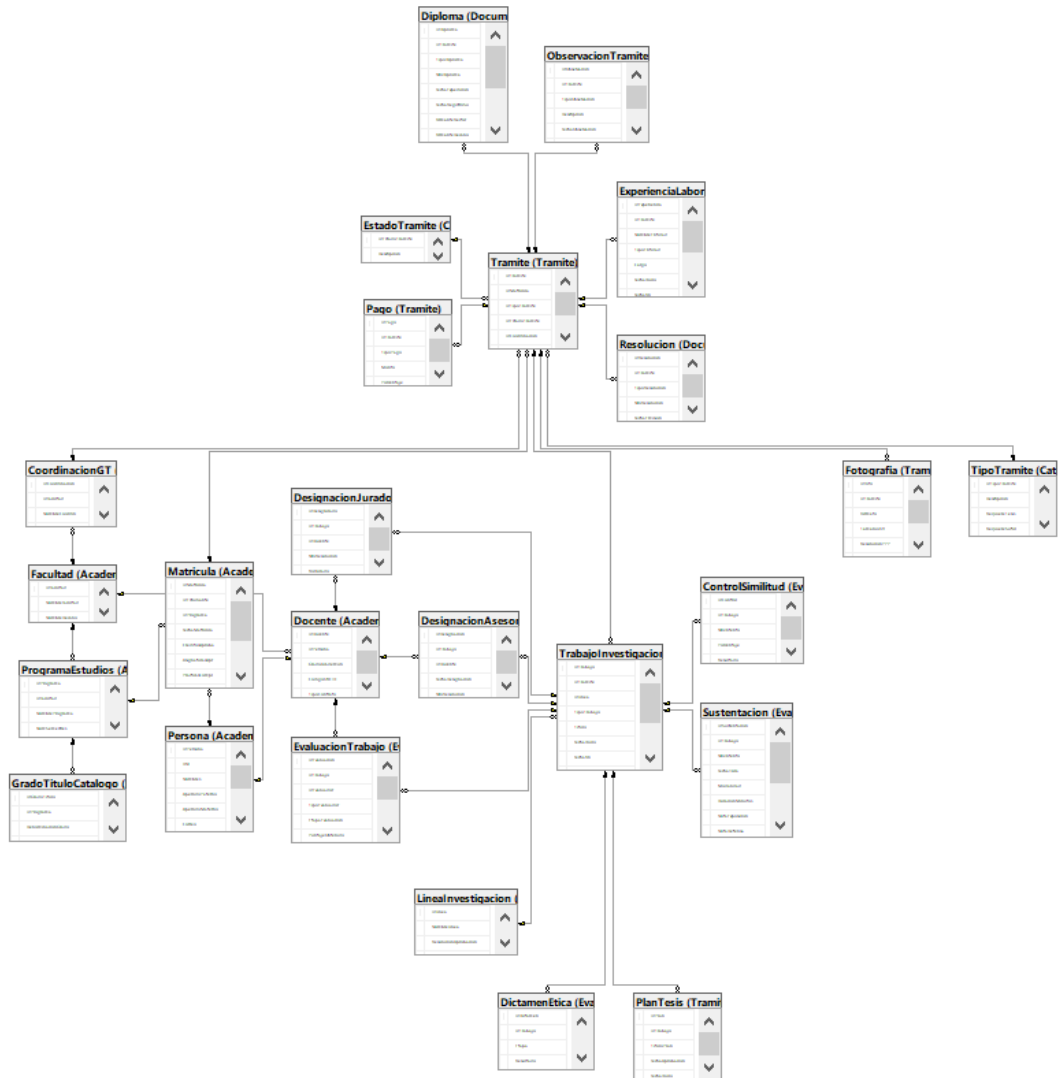
Anexo A. Resumen técnico del script proporcionado

El archivo fuente utilizado para elaborar este informe comienza con USE GradosTitulos y posteriormente define los esquemas de la solución. La documentación se elaboró a partir del script proporcionado.

```
USE GradosTitulos;
GO

CREATE SCHEMA Catalogo;
GO
CREATE SCHEMA Academico;
```

```
GO
CREATE SCHEMA Tramite;
GO
CREATE SCHEMA Evaluacion;
GO
CREATE SCHEMA Documento;
GO
```



```

USE GradosTitulos;
GO

-- =====
-- 2. CREAR ESQUEMAS
-- =====
CREATE SCHEMA Catalogo; -- Tablas de parámetros y catálogos
GO
CREATE SCHEMA Academico; -- Entidades académicas principales
GO
CREATE SCHEMA Tramite; -- Proceso administrativo
GO
CREATE SCHEMA Evaluacion; -- Evaluaciones y sustentación
GO
CREATE SCHEMA Documento; -- Resoluciones, diplomas y documentos
GO

-- =====
-- 3. TABLAS DE CATÁLOGO
-- =====

-- T01: TIPO_TRAMITE
-- Catálogo de tipos de trámite disponibles

```

100 %

Results Messages

	Esquema	Tabla	NroColumnas	NroIndices
1	Academico	Docente	21	21
2	Academico	Facultad	8	8
3	Academico	GradoTituloCatalogo	5	5
4	Academico	Matricula	33	33
5	Academico	Persona	36	36
6	Academico	ProgramaEstudios	18	18
7	Catalogo	EstadoTramite	6	6
8	Catalogo	LineaInvestigacion	10	10
9	Catalogo	TipoTramite	12	12
10	Documento	Diploma	24	24
11	Documento	Resolucion	28	28

	Esquema	Vista
1	Academico	V_CondicionEgresado
2	Evaluacion	V_NotaFinalSustentacion
3	Tramite	V_EstadoTrabajo

LAB04-PC14\MSSQLSERVERDEV (SQL Server 10.0.1000.0 - LAB04-PC14)

Databases

- System Databases
- Database Snapshots
- GradosTitulos
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - Academico.Docente
 - Academico.Facultad
 - Academico.GradoTituloCatalogo
 - Academico.Matricula
 - Academico.Persona
 - Academico.ProgramaEstudios
 - Catalogo.EstadoTramite
 - Catalogo.LineaInvestigacion
 - Catalogo.TipoTramite
 - Documento.Diploma
 - Documento.Resolucion
 - Evaluacion.ControlSimilitud
 - Evaluacion.DictamenEtica
 - Evaluacion.EvaluacionTrabajo
 - Evaluacion.Sustentacion
 - Tramite.CoordinacionGT
 - Tramite.DesignacionAsesor
 - Tramite.DesignacionJurado
 - Tramite.ExperienciaLaboral
 - Tramite.Fotografia
 - Tramite.ObservacionTramite
 - Tramite.Pago
 - Tramite.PlanTesis
 - Tramite.TrabajoInvestigacion
 - Tramite.Tramite
 - Dropped Ledger Tables
- Views
 - System Views
 - Academico.V_CondicionEgresado
 - Evaluacion.V_NotaFinalSustentacion
 - Tramite.V_EstadoTrabajo
 - Columns
 - Triggers
 - Indexes
 - Statistics

CREATE TABLE

```
CREATE TABLE [GradosTitulos].[Academico].[Tramite] (
    IdEstudiante INT,
    Descripcion VARCHAR(255),
    EstadoTramite VARCHAR(50),
    CONSTRAINT PK_Tramite PRIMARY KEY (IdEstudiante),
    CONSTRAINT FK_Tramite FOREIGN KEY (IdEstudiante) REFERENCES [GradosTitulos].[Academico].[Persona] (IdPersona)
);
GO
```

INSERT INTO

```
INSERT INTO [GradosTitulos].[Academico].[Tramite] (IdEstudiante, Descripcion, EstadoTramite)
VALUES (1, 'INIC', 'INIC'),
(2, 'EN_V', 'EN_V'),
(3, 'EN_F', 'EN_F'),
(4, 'OBSE', 'OBSE'),
(5, 'APRC', 'APRC'),
(6, 'IMPR', 'IMPR'),
(7, 'ARCH', 'ARCH'),
(8, 'SUSP', 'SUSP');
GO
```

100 %

Results

	Esquema
15	Evaluacion
16	Tramite
17	Tramite
18	Tramite
19	Tramite
20	Tramite
21	Tramite
22	Tramite
23	Tramite
24	Tramite
25	Tramite

	Esquema
1	Academico
2	Evaluacion
3	Tramite